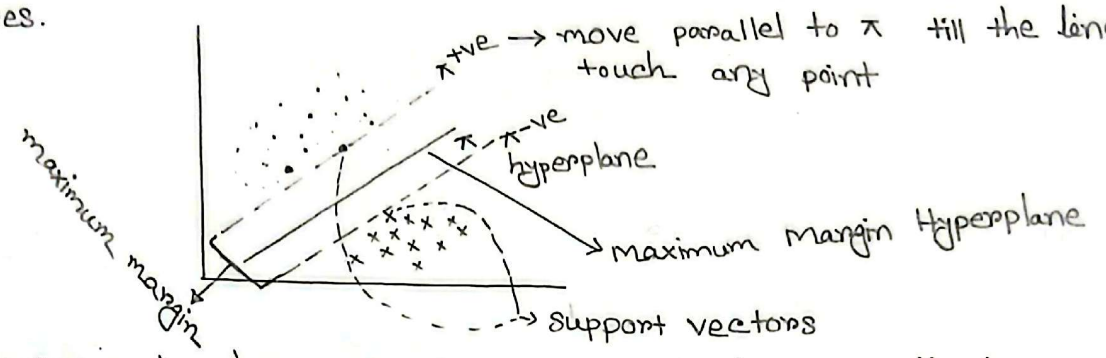
Support Vector Machines

- a supervised machine learning algorithm used for both classification and regression.

→ It tries to find the best boundary known as hyperplane that seperates different classes.



#Hyperplane: A decission boundary separating different classes in feature space.

Support vector: closest points to the hyper plane.

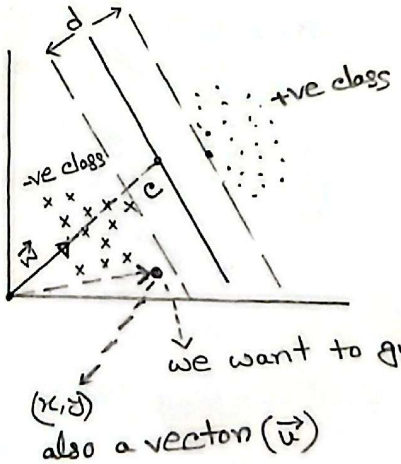
#margin: Distance between hyperplane and support vectors.
ⓓ

① Take a hyperplane (π)
② Find +ve and -ve hyperplane
③ Find the margin(d)
④ Take the hyperplane for which margin is maximum.

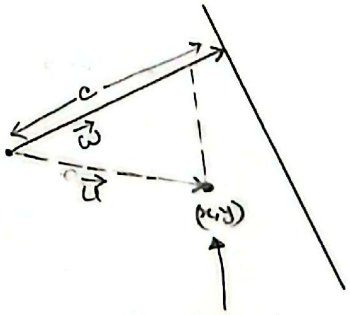
equation for hyperplane is: $\boxed{w^T x + b = 0}$

Decission Rule

$\vec{w}$ must be perpendicular with ↑



$\vec{w}$ → Perpendicular to all ∞ lines



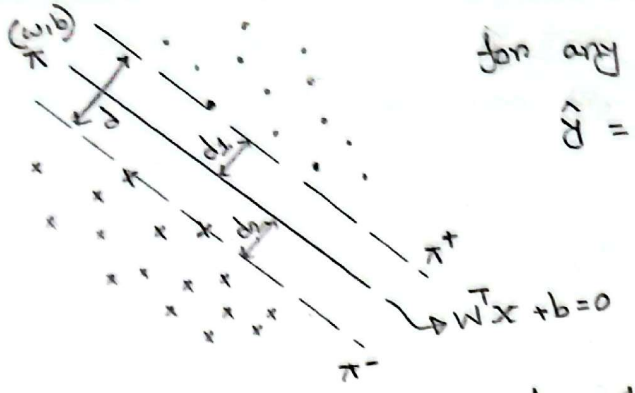
To classify this point take the projection $\vec{u}$ on $\vec{w}$

If Projection crosses the decission boundary then +ve else -ve

Projection

$\vec{w}.\vec{u} \geq c$

$\Rightarrow \vec{w}.\vec{u} - c \geq 0 \Rightarrow \vec{w}.\vec{u} + b \geq 0$ $[b = -c]$



for any point $\vec{x}_i$

$$\hat{y} = \begin{cases} +1 \text{ if } \vec{w}.\vec{x} + b \geq 0 \\ -1 \text{ if } \vec{w}.\vec{x} + b < 0 \end{cases}$$

Date : .....

we want to maximize the value of d for the equation $w^T x + b = 0$

let's assume equation for $\pi^{+ve} \Rightarrow w^T x + b = 1$

$\pi^{-ve} \Rightarrow w^T x + b = -1$

} why? cause while calculating the distance d_1 and d_2 always need to be equal.

+ve π : $w^T x + b = 1$

π : $w^T x + b = 0$

-ve π : $w^T x + b = -1$

Notes: If we multiply LHS of +ve π and -ve π with any positive value then margin(d) shrinks.

If we divide with +ve value then margin expand.

while calculating d there is a constraints that +ve points can't cross π^+ and -ve points can't cross π^-

For all +ve class $w^T x + b \geq 1$ → For support vector $w^T x + b = 1$

" other point $w^T x + b > 1$

For all -ve class $w^T x + b \leq -1$

If this two constraints are satisfied then we will calculated d and try to maximize.

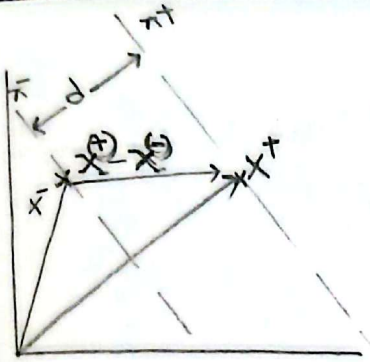
+ is label of +ve class → $+(w^T x + b) \geq 1$

$-(w^T x + b) \geq -(-1)$ ← label of -ve class

→ multiplying 2nd equation with -1 will invert the relation

$y_i (w^T x + b) \geq 1$

↑ can be +ve or -ve class



To find the distance d we have to take the Projection of $x^{(+)} - x^{(-)}$ on the $\vec{w}$

$$d = (\vec{x}^+ - \vec{x}^-) \cdot \frac{\vec{w}}{|\vec{w}|}$$

$$= \frac{\vec{x}^{(+)}\vec{w} - \vec{x}^{(-)}\vec{w}}{|\vec{w}|}$$

now as x^+ and x^- both are support vector so they lie on π^+ and π^- means for x^+ and x^- $\boxed{y_i(\vec{w}\vec{x}_i + b) = 1}$ is true

for x^+, $y_i = 1$ so $\vec{w}\vec{x}_i + b = 1$

$\Rightarrow \vec{w}\vec{x}_i = 1 - b$

for $\vec{x}^{(-)}$, $y_i = -1$

$-(\vec{w}\vec{x} + b) = 1$

$\Rightarrow \vec{w}\vec{x} = -1 - b$

$$d = \frac{1 - b - (-1 - b)}{||w||}$$

$$\Rightarrow d = \frac{1 - b + 1 + b}{||w||} = \frac{2}{||w||}$$

$\boxed{d = \frac{2}{||w||}}$ → we have to maximize this

$$\text{argmax}\ (w^*, b^*) = \frac{2}{|w|} \quad \text{such that}$$

$$y_i(w^T x_i + b) \geq 1$$

Note: As this function will fail if classes are not purely separable that's why its called hard-margin SVM

maximizing $\frac{1}{||w||}$ $\Rightarrow$ minimize $|w|$ $\Rightarrow$ minimize $\frac{1}{2}||w||^2$

Basically we are trying to find the extreme of a function with some constraints we have to use Lagrange Multipiler.

$$L = \frac{1}{2}||\vec{w}||^2 - \sum_{i=1}^{n} \alpha_i \left[y_i(\vec{w}.\vec{x}_i + b) - 1 \right]$$

$n \rightarrow$ num of train data

$\alpha_i \geq 0 \rightarrow$ Lagrange multipiler

we want to find the value of $\vec{w}$, so the L minimized and b

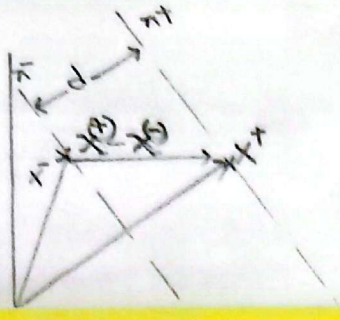
$$\frac{\partial L}{\partial \vec{w}} = \vec{w} - \sum \alpha_i y_i \vec{x}_i$$

$$\vec{w} - \sum \alpha_i y_i \vec{x}_i = 0$$

$$\Rightarrow \boxed{\vec{w} = \sum \alpha_i y_i \vec{x}_i}$$

$$\frac{\partial L}{\partial b} = -\sum_i \alpha_i y_i$$

$$\boxed{\sum \alpha_i y_i = 0}$$



To find the distance d we have to take the Projection of $x^{(+)} - x^{(-)}$ on the $\vec{w}$

$$d = (\vec{x}^+ - \vec{x}^-) \cdot \frac{\vec{w}}{|\vec{w}|}$$

$$= \frac{\vec{x}^{(+)}\vec{w} - \vec{x}^{(-)}\vec{w}}{|\vec{w}|}$$

x^+ and x^- both are support vector so they lie on gutters for x^+ and x^- $\left[y_i(\vec{w}\vec{x}_i + b) = 1\right]$ is true

$\vec{w}\vec{x}^{(+)} + b = 1$

$\vec{w}\vec{x}^{(+)} = 1 - b$ for $\vec{x}^{(-)}$, $y_i = -1$

$-(\vec{w}\vec{x} + b) = 1$

$\Rightarrow \vec{w}\vec{x} = -1 - b$

$$\boxed{d = \frac{2}{\|w\|}} \rightarrow \text{we have to maximize this}$$

$$\text{argmax}\,(w^*, b^*) = \frac{2}{|w|} \quad \text{such that} \quad y_i(w^T x_i + b) \geq 1$$

will fail if classes are not purely separable
hard-margin SVM

maximizing $\frac{1}{\|w\|} \Rightarrow$ minimize $|w| \Rightarrow$ minimize $\frac{1}{2}\|w\|^2$

Basically we are trying to find the extreme of a function with some constraints we have to use Lagrange Multipiler.

$$L = \frac{1}{2}\|\vec{w}\|^2 - \sum_{i=1}^{n} \alpha_i \left[y_i(\vec{w}.\vec{x}_i + b) - 1\right]$$

$n \rightarrow$ num of train data

$\alpha_i \geq 0 \rightarrow$ Lagrange multipiler

we want to find the value of $\vec{w}$ and b so the L minimized

$$\frac{\partial L}{\partial \vec{w}} = \vec{w} - \sum \alpha_i y_i \vec{x}_i \qquad \frac{\partial L}{\partial b} = -\sum_i \alpha_i y_i$$

$$\vec{w} - \sum \alpha_i y_i \vec{x}_i = 0$$

$$\Rightarrow \boxed{\vec{w} = \sum \alpha_i y_i \vec{x}_i} \qquad \boxed{\sum \alpha_i y_i = 0}$$

$\frac{1}{2}\|w\|^2$ subject to $y_i(w^T x_i + b) \geq 1$, $i = 1 \ldots m$ → no of train data

constraint $\Rightarrow g_i(w,b) = 1 - y_i(w^T x_i + b) \leq 0$

$\therefore = \frac{1}{2}\|w\|^2 + \sum \alpha_i (1 - y_i(w^T x_i + b))$

put $\vec{w}$ and b in L

$$L = \frac{1}{2}\left(\sum_i \alpha_i y_i \bar{x}_i\right)\left(\sum_j \alpha_j y_j \bar{x}_j\right) - \left(\sum \alpha_i y_i \bar{x}_i\right)\left(\sum \alpha_j y_j \bar{x}_j\right) - \sum \alpha_i y_i b + \sum \alpha_i$$

$$L = \sum \alpha_i - \frac{1}{2}\left(\sum_i \alpha_i y_i \bar{x}_i\right)\left(\sum_j \alpha_j y_j \bar{x}_j\right) - b \underbrace{\sum \alpha_i y_i}_{0}$$

$$\boxed{L = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \bar{x}_i \bar{x}_j}$$ → now we have to find the extreme valu

optimization depends on only this dot Product pair

we did all these things to find what this maximization depends on with respect these vector

now decision $\sum \alpha_i y_i \boxed{\bar{x}_i \cdot \vec{u}} + b \geq 0$ then +ve class else -ve

$\vec{u}$: unknown point that want to predicd

$\bar{x}_i$: decision is also depends only the dot Product

so for linearly not separable point we have to transform the plane where it will be separable.

As the optimization only depends on dot product so we will transform the depended part only.

$K(\bar{x}_i, \bar{x}_j) = \phi(\bar{x}_i) \cdot \phi(\bar{x}_j)$ need to max

so now we don't need to do the transformation

* dual optimization Problem $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$

subject to $\alpha_i \geq 0$ for all $i = 1 \ldots n$, $\sum_{i=1}^{n} \alpha_i y_i = 0$

its a Quadratic Programming problem.

by solving this we will get α^* the optimal α where $\alpha_i \geq 0$

now Recover the weight vector and bias b

$$w^* = \sum_{i=1}^{m} \alpha_i^* y_i x_i$$

Picking any support vector (x_k, y_k) with $\alpha_k > 0$

using condition

$$y_k(w^{*T} x_k + b^*) = 1$$

$$\Rightarrow \boxed{b^* = y_k - w^{*T} x_k}$$

final classifier $f(x) = \text{sign}\left(w^{*T} x + b^*\right)$

$$= \text{sign}\left(\sum_{i=1}^{m} \alpha_i^* y_i \underbrace{(x_i^T x)} + b^*\right)$$

main optimization term

Why kernel

$f(x) = \text{sign}(w^{*T} + b^*)$ is only work when train data is only linearly seperable. (Real world don't have that).

so If we map these data points through a transformer $\phi(x)$

$$\phi : R^n \longrightarrow R^p \quad [p >> n]$$

then in that higher dimension the data might become linearly separable.

If $\phi(x)$ has $1000/\infty$ dimension computing is not possible.

as optimization only depends on $x_i^T x_j$ so in transformed space we only need $\phi(x_i)^T \phi(x_j)$

we use kernel trick $K(x_i, x_j) = \phi(x_i)^T \cdot \phi(x_j)$ without ever caculating $\phi(x)$ explicitly.

common strategis for multi-class

Date : .....

ⓐ One v/s Others (OvR)

→ Train k number of classifier for k class.
→ For class k : train a svm where k = +1, all other = -1
→ For prediction pick the class whose have largest margin score.

ⓑ One vs one (OvO)

Train a classifier for each pair of class
If total k class train $\frac{k(k-1)}{2}$ models
For Prediction each classifier votes and majority wins.